

```
!pip install -q opencv-python numpy matplotlib scikit-image
```

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from skimage.morphology import (
    opening, closing, remove_small_objects, remove_small_holes,
    disk, square
)
from skimage import io, img_as_bool
```

```
image_paths = [
    "/content/Picture1.png",
    "/content/Picture2.png",
    "/content/Picture3.png"
]

images = []
for p in image_paths:
    img = io.imread(p, as_gray=True)
    binary = img > 0.5 # convert to binary
    images.append((p, binary))
    print("Loaded:", p)
```

```
Loaded: /content/Picture1.png
Loaded: /content/Picture2.png
Loaded: /content/Picture3.png
```

```
def process_binary_image(binary_img,
                          speck_size=40,
                          crack_size=60,
                          se_small=3,
                          se_large=5):
    """
    speck_size: maximum small bright noise size
    crack_size: maximum dark crack thickness
    se_small: size for removing noise (opening)
    se_large: size for closing cracks
    """

    # Structuring elements
    se_open = square(se_small)
    se_close = square(se_large)

    # Step 1: Remove small bright specks
    cleaned = remove_small_objects(binary_img, min_size=speck_size)

    # Step 2: Smooth noise with opening (erosion → dilation)
    opened = opening(cleaned, se_open)

    # Step 3: Fill thin dark cracks using closing (dilation → erosion)
    closed = closing(opened, se_close)

    # Step 4: Ensure cracks fully filled by removing small holes
    final = remove_small_holes(closed, area_threshold=crack_size)

    return cleaned, opened, closed, final
```

```
results = {}

for path, img in images:
    cleaned, opened, closed, final = process_binary_image(img,
                                                           speck_size=40,
                                                           crack_size=60,
                                                           se_small=3,
                                                           se_large=5)

    results[path] = (img, cleaned, opened, closed, final)
    print("Processed:", path)
```

```
Processed: /content/Picture1.png
Processed: /content/Picture2.png
Processed: /content/Picture3.png
/tmp/ipython-input-672278072.py:14: FutureWarning: `square` is deprecated since version 0.25 and will be removed
    se_open = square(se_small)
/tmp/ipython-input-672278072.py:15: FutureWarning: `square` is deprecated since version 0.25 and will be removed
    se_close = square(se_large)
```

```
for path, (orig, cleaned, opened, closed, final) in results.items():
```

```
plt.figure(figsize=(12,8))
plt.suptitle(f"Processing Results: {path}")

plt.subplot(2,3,1)
plt.imshow(orig, cmap="gray")
plt.title("Original")
plt.axis("off")

plt.subplot(2,3,2)
plt.imshow(cleaned, cmap="gray")
plt.title("Small Bright Specks Removed")
plt.axis("off")

plt.subplot(2,3,3)
plt.imshow(opened, cmap="gray")
plt.title("After Opening")
plt.axis("off")

plt.subplot(2,3,4)
plt.imshow(closed, cmap="gray")
plt.title("After Closing")
plt.axis("off")

plt.subplot(2,3,5)
plt.imshow(final, cmap="gray")
plt.title("Final Output")
plt.axis("off")

plt.tight_layout()
plt.show()
```

Start coding or [generate](#) with AI.